

Unit 4: Inheritance

Prof. Mahipal Khoja

Assistant Professor

Artificial Intelligence and Data Science Department

Content

1. Introduction.....	3
2. Defining Derived Classes.....	15
3. Types of inheritance.....	16
4 . Single Inheritance	17
5. Multiple Inheritance	20
6. Multilevel Inheritance	22
7. Hybrid Inheritance	26
8. Function Overriding	28
9. Function Overloading	31
12. Function Overloading vs. Function Overriding.....	33
13. Virtual Base Class	34
14. Abstract classes	34

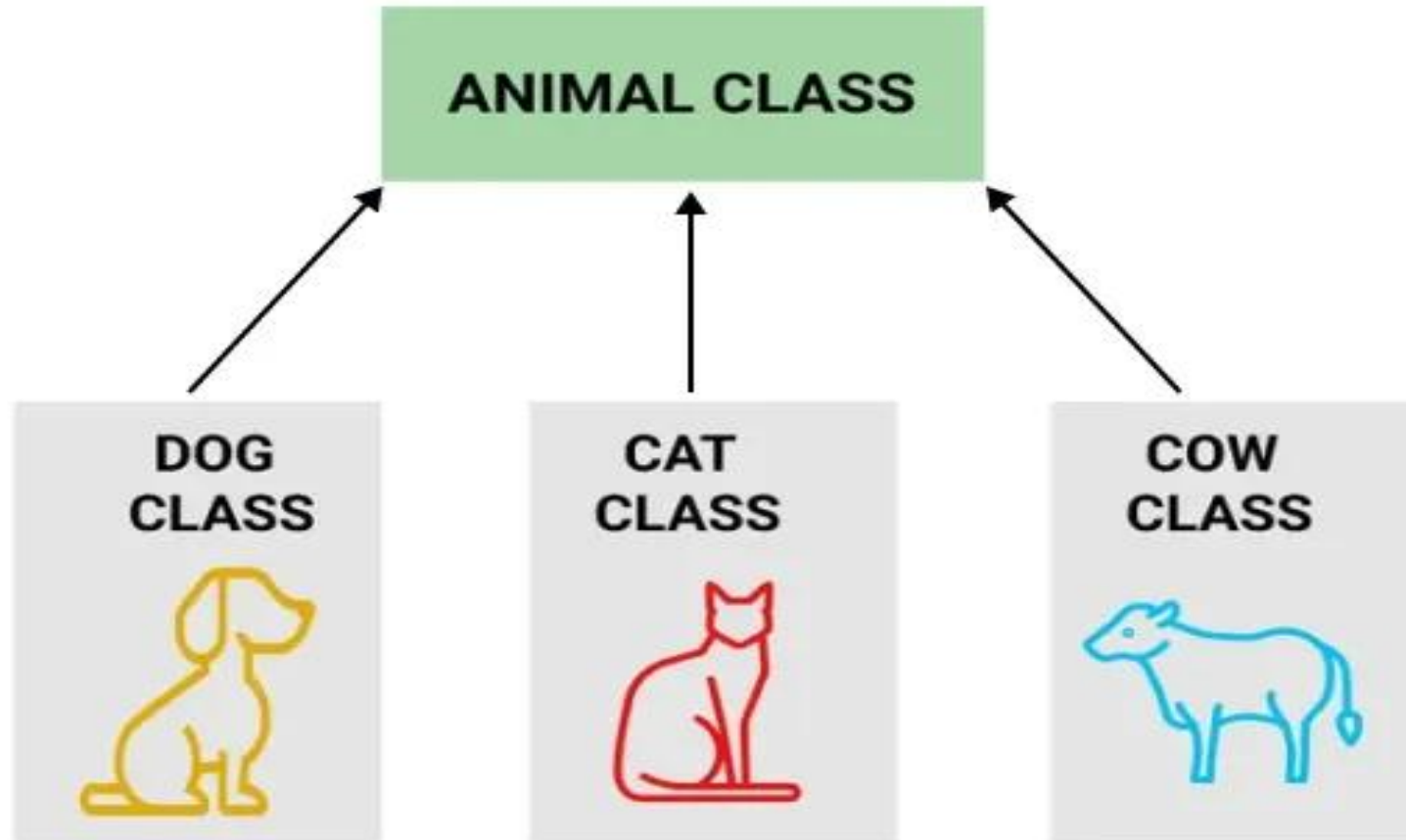
Introduction

➤ Inheritance is a fundamental concept in OOP (Object Oriented Programming). It is the mechanism by which one class is allowed to inherit the features (fields and methods) of another class.

Inheritance means creating new classes based on existing ones.

A class that inherits from another class can reuse the methods and fields of that class.

➤ In the following example, Animal is the base class and Dog, Cat and Cow are derived classes that extend the Animal class.



```
#include <iostream>
using namespace std;
class Animal
{
public:
    void sound()
    {
        cout << "Animal makes
a sound" << endl;
    }
};
class Dog : public Animal
{
```

```
public:
    void sound()
    {
        cout << "Dog barks" << endl;
    }
};
class Cat : public Animal
{
public:
    void sound()
    {
        cout << "Cat meows" << endl;
    }
};
```

```
class Cow : public Animal
{
    public:
        void sound()
        {
            cout << "Cow moos" << endl;
        }
};
```

```
int main()
{
    Dog d;
    d.sound();
    Cat c;
    c.sound();
    Cow cow;
    cow.sound();
    return 0;
}
```

Output

Dog barks

Cat meows

Cow moos

Animal is the base class with a function sound().

Dog, Cat, and Cow are derived classes, each defining their own sound() method.

In main(), objects of Dog, Cat, and Cow are created separately. When we call sound() on each object, the respective child class method runs (Dog barks, Cat meows, Cow moos).

Syntax

```
class ChildClass : public ParentClass  
{  
    // Additional fields and methods  
};
```

How Inheritance Works in C++?

The colon (:) with an access specifier is used for inheritance in C++. It allows the derived class (child class) to inherit the data members (fields) and member functions (methods) of the base class (parent class).

When a class inherits another class, it gets all the accessible members of the parent class, and the child class can also redefine (override) or add new functionality to them.

Difference between Base class and Derived class in C++

Base Class: A base class is a class in Object-Oriented Programming language, from which other classes are derived. The class which inherits the base class has all members of a base class as well as can also have some additional properties. The Base class members and member functions are inherited to Object of the derived class. A base class is also called parent class or superclass.

Derived Class: A class that is created from an existing class. The derived class inherits all members and member functions of a base class. The derived class can have more functionality with respect to the Base class and can easily access the Base class. A Derived class is also called a child class or subclass.

```
class BaseClass{  
    // members....  
    // member function  
}
```

```
class DerivedClass : public  
BaseClass{  
    // members....  
    // member function  
}
```

S.No.	BASE CLASS	DERIVED CLASS
1.	A class from which properties are inherited.	A class from which is inherited from the base class.
2.	It is also known as parent class or superclass.	It is also known as child class subclass.
3.	It cannot inherit properties and methods of Derived Class.	It can inherit properties and methods of Base Class.
4.	Syntax: class BaseClass{ // members.... // member function }	Syntax: class DerivedClass : access_specifier BaseClass{ // members.... // member function

```
// C++ program to illustrate  
// Base & Derived Class
```

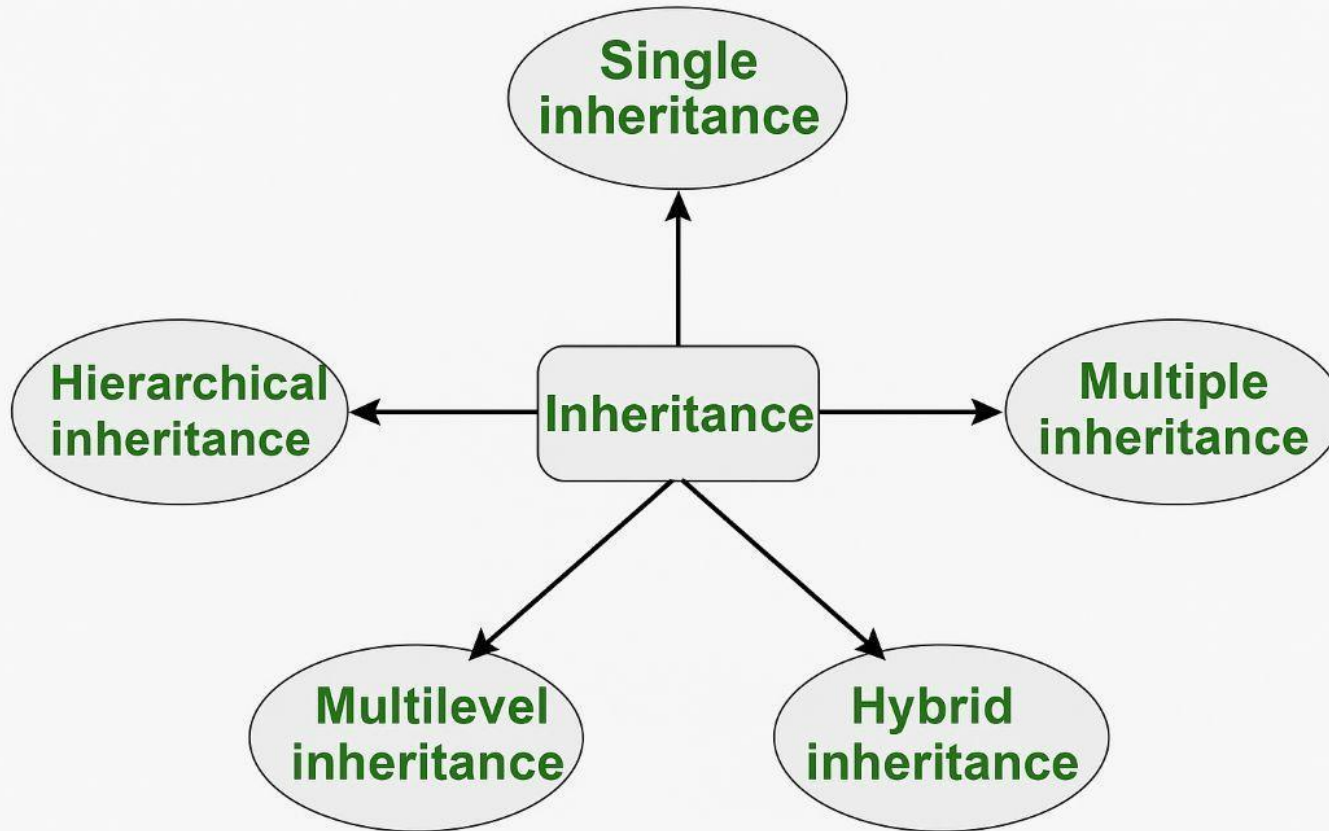
```
#include <iostream>  
using namespace std;
```

```
// Declare Base Class  
class Base {  
public:  
    int a;  
};
```

```
// Declare Derived Class  
class Derived : public Base {  
public:  
    int b;  
};
```

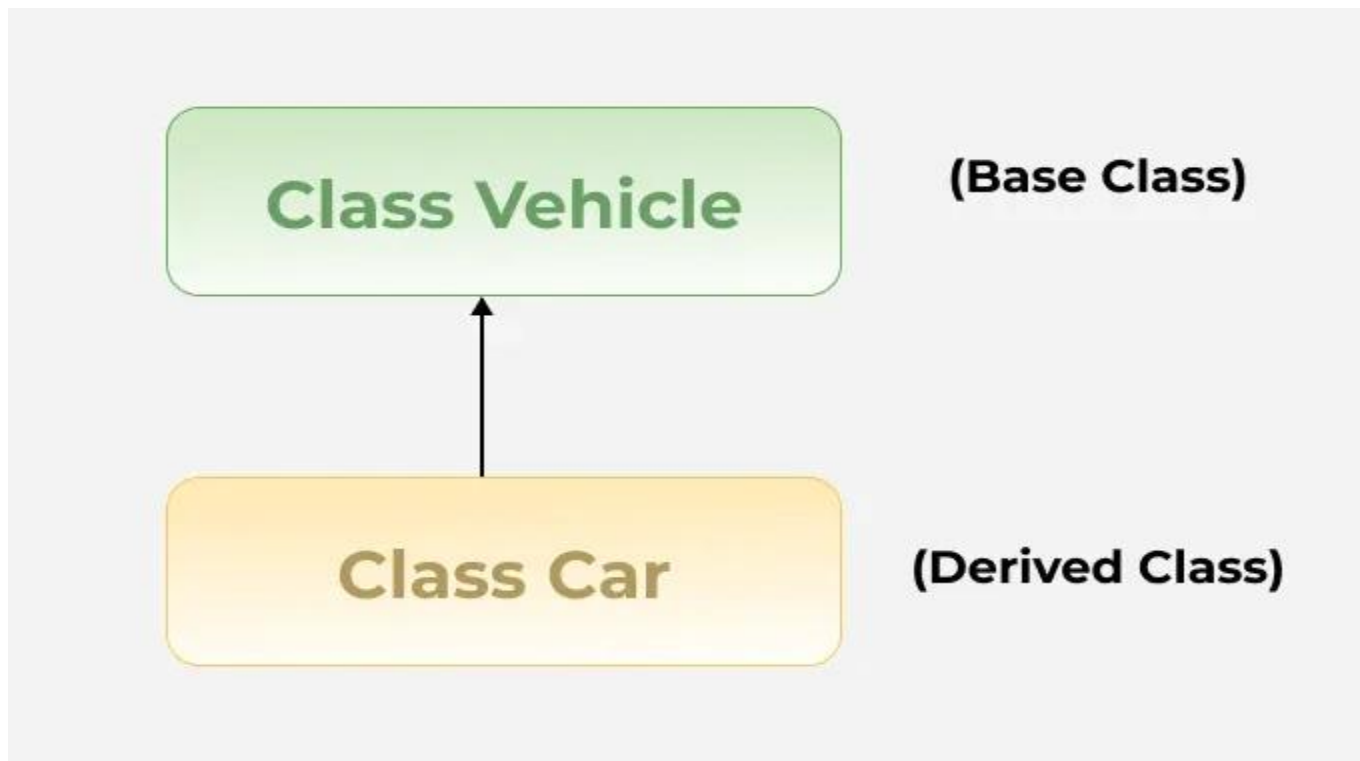
```
// Driver Code  
int main()  
{  
    // Initialise a Derived class  
    Derived geeks;  
    // Assign value to Derived class variable  
    geeks.b = 3;  
  
    // Assign value to Base class variable  
    // via derived class  
    geeks.a = 4;  
  
    cout << "Value from derived class: "  
        << geeks.b << endl;  
    cout << "Value from base class: "  
        << geeks.a << endl;  
  
    return 0;  
}
```

Types of Inheritance in C++



Single Inheritance

In single inheritance, a sub-class is derived from only one super class. It inherits the properties and behavior of a single-parent class. Sometimes, it is also known as simple inheritance.



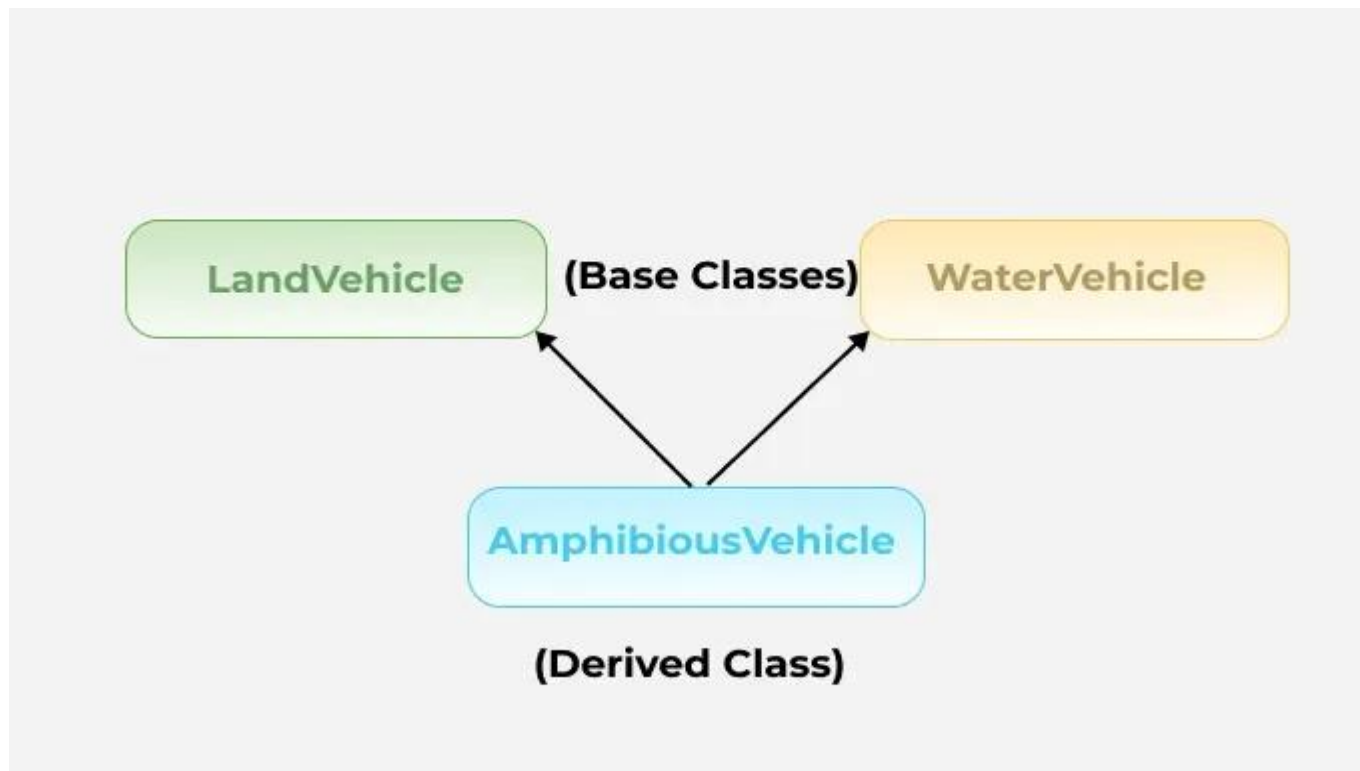
// Single Inheritance

```
#include <iostream>
using namespace std;
class Vehicle {
public:
    Vehicle() {
        cout << "This is a Vehicle" << endl;
    }
};
class Car : public Vehicle {
public:
    Car() {
        cout << "This Vehicle is Car" << endl;
    }
};
```

```
int main() {
    Car obj;
    return 0;
}
```

Multiple Inheritance

In Multiple inheritance, one class can have more than one superclass and inherit features from all parent classes.



Multiple Inheritance

```
#include <iostream>
using namespace std;
class LandVehicle
{
public:
    void landInfo()
    {
        cout << "This is a LandVehicle" << endl;
    }
};
class WaterVehicle
{
public:
    void waterInfo()
    {
        cout << "This is a WaterVehicle" << endl;
    }
};
```

```
// Derived class inheriting from both base classes
class AmphibiousVehicle : public LandVehicle, public
WaterVehicle
{
    public:
        AmphibiousVehicle()
        {
            cout << "This is an AmphibiousVehicle" << endl;
        }
};

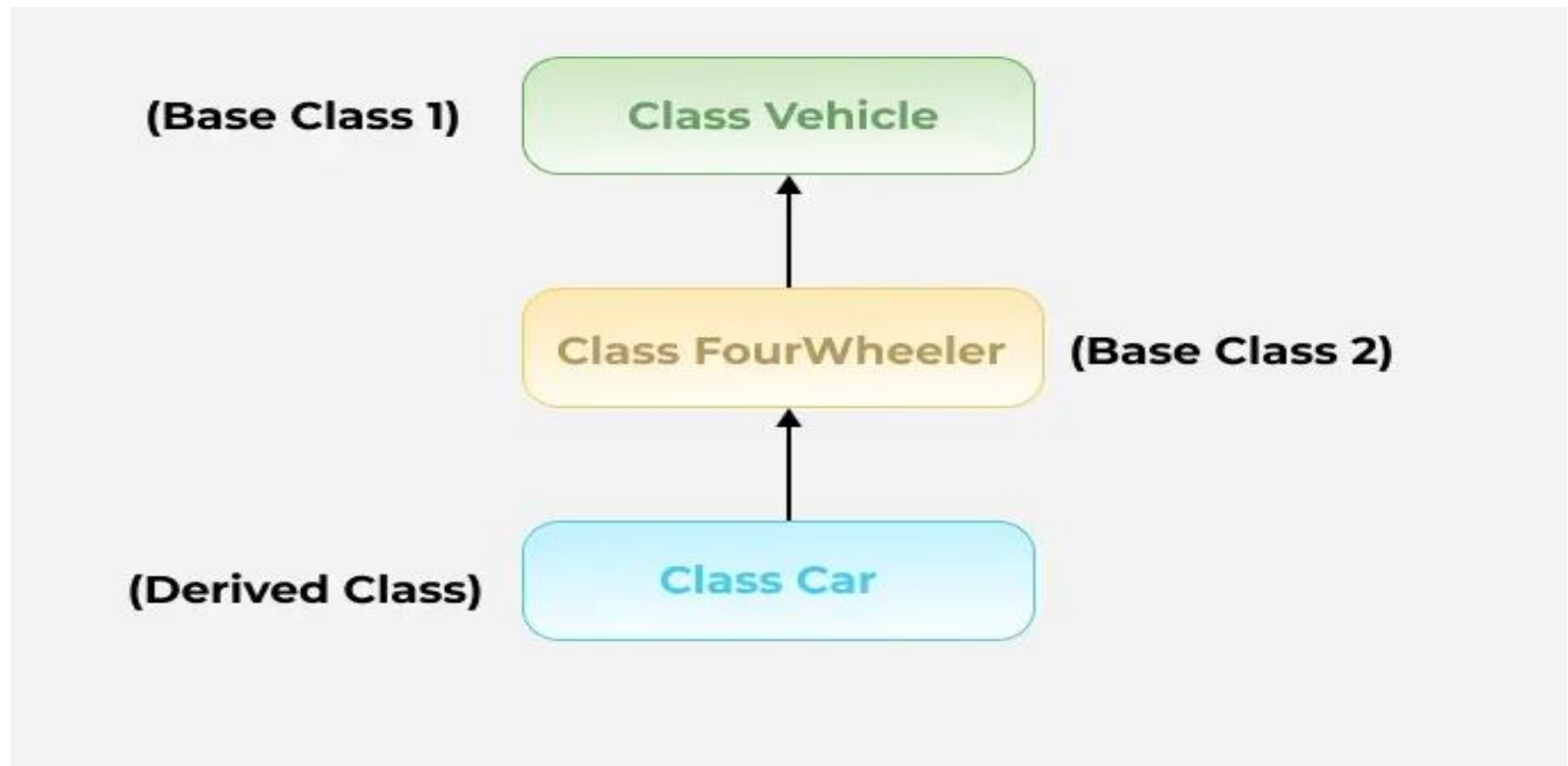
int main()
{
    AmphibiousVehicle obj;
    obj.waterInfo();
    obj.landInfo();
    return 0;
}
```

Output

This is an AmphibiousVehicle
This is a WaterVehicle
This is a LandVehicle

Multilevel Inheritance

Multilevel inheritance in C++ means a class is derived from another derived class, forming a chain of inheritance.



```
#include <iostream>
using namespace std;

class Vehicle
{
public:
    Vehicle()
    {
        cout << "This is a Vehicle" << endl;
    }
};

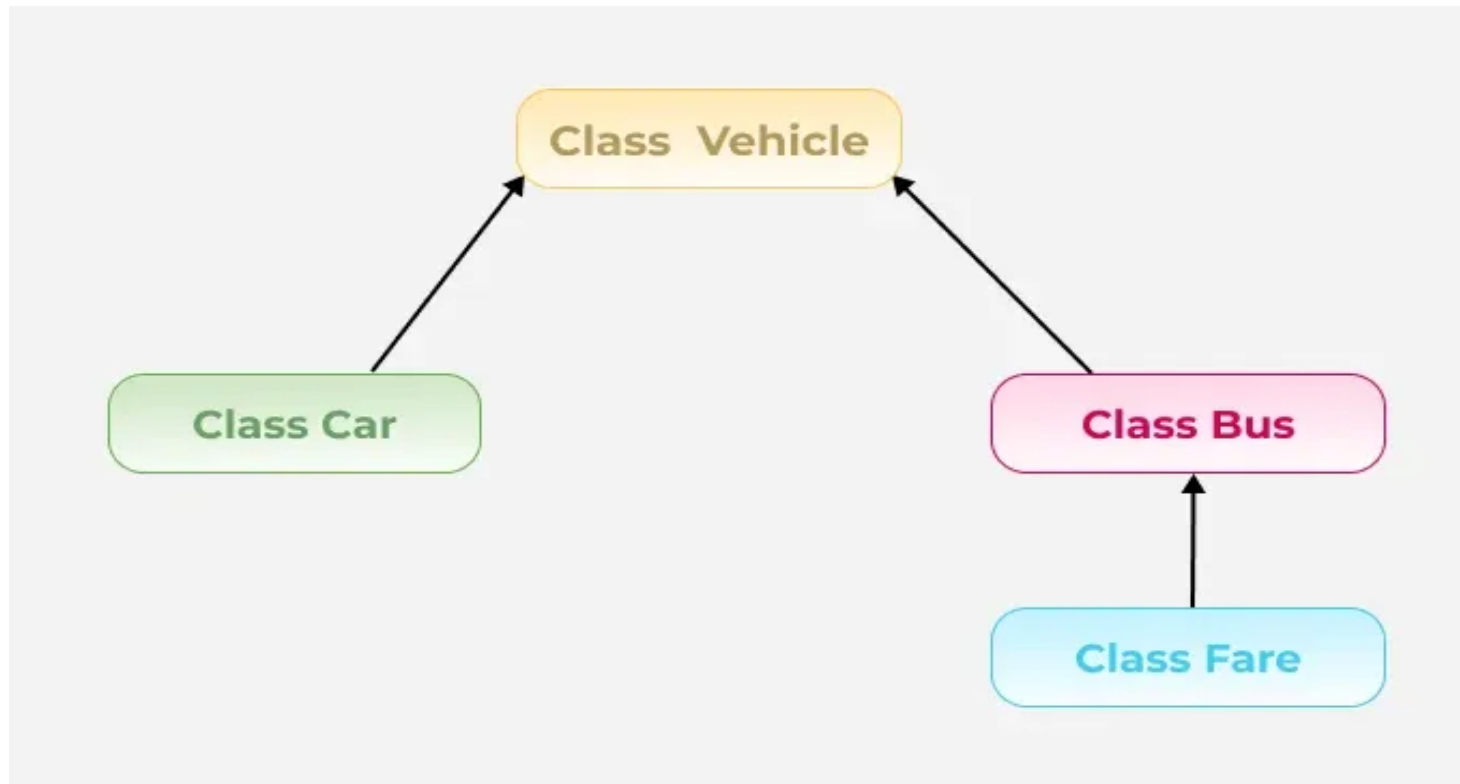
// Derived class from Vehicle
class FourWheeler : public Vehicle
{
public:
    FourWheeler()
    {
        cout << "4 Wheeler Vehicles" << endl;
    }
};
```

```
// Derived class from FourWheeler
class Car : public FourWheeler
{
public:
    Car()
    {
        cout << "This 4 Wheeler Vehicle is a Car"
        << endl;
    }
};

int main()
{
    Car obj;
    return 0;
}
```

Hybrid Inheritance

When two or more types of inheritance are combined in one program. For example, a class might use multiple inheritance and also be part of a multilevel inheritance chain.



```
#include <iostream>
using namespace std;

class Vehicle
{
public:
    Vehicle()
    {
        cout << "This is a Vehicle" <<
endl;
    }
};

class Fare
{
public:
    Fare()
    {
        cout << "Fare of Vehicle" << endl;
    }
};

class Car : public Vehicle
{
public:
    Car()
    {
        cout << "This Vehical is a Car" <<
endl;
    }
};
```

```
class Bus : public Vehicle, public Fare
{
    public:
        Bus()
        {
            cout << "This Vehicle is a Bus with
Fare";
        }
};

int main()
{
    Bus obj2;
    return 0;
```

Output

This is a Vehicle

Fare of Vehicle

This Vehicle is a Bus with
Fare

Function Overloading (achieved at compile time)

Function Overloading provides multiple definitions of the function by changing signature i.e. changing number of parameters, change datatype of parameters, return type doesn't play any role.

It can be done in base as well as derived class

```
// CPP program to illustrate
// Function Overloading
#include <iostream>
using namespace std;
// overloaded functions
void test(int);
void test(float);
void test(int, float);
int main()
{
    int a = 5;
    float b = 5.5;
    // Overloaded functions
    // with different type and
    // number of parameters
```

```
test(a);
test(b);
test(a, b);
return 0;
}
// Method 1
void test(int var)
{
    cout << "Integer number: " << var <<
endl;
}
```

```
// Method 2
void test(float var)
{
    cout << "Float number: " << var <<
endl;
}
```

```
// Method 3
void test(int var1, float var2)
{
    cout << "Integer number: " << var1;
    cout << " and float number:" <<
var2;
}
```

Output

Integer number: 5

Float number: 5.5

Integer number: 5 and float
number:5.5

Function Overriding (achieved at run time)

It is the redefinition of base class function in its derived class with same signature i.e. return type and parameters.

It can only be done in derived class.

```
Class a
{
public:
    virtual void display(){ cout << "hello"; }
};
```

```
Class b:public a
{
public:
    void display(){ cout << "bye";}
};
```

```
// CPP program to illustrate
// Function Overriding
#include<iostream>
using namespace std;

class BaseClass
{
public:
    virtual void Display()
    {
        cout << "\nThis is Display() method"
              " of BaseClass";
    }
}
```

```
void Show()
{
    cout << "\nThis is Show() method "
          "of BaseClass";
}

};

class DerivedClass : public BaseClass
{
public:
    // Overriding method - new working of
    // base class display method
    void Display()
    {
        cout << "\nThis is Display() method"
              " of DerivedClass";
    }
};
```

// Driver code

int main()

{

DerivedClass dr;

BaseClass &bs = dr;

bs.Display();

dr.Show();

}

Output

This is Display() method of DerivedClass

This is Show() method of BaseClass

Function Overloading	Function Overriding
Function Overloading provides multiple definitions of the function by changing signature.	Function Overriding is the redefinition of base class function in its derived class with same signature.
An example of compile time polymorphism.	An example of run time polymorphism.
Function signatures should be different.	Function signatures should be the same.
Overloaded functions are in same scope.	Overridden functions are in different scopes.

Function Overloading	Function Overriding
Overloading is used when the same function has to behave differently depending upon parameters passed to them.	Overriding is needed when derived class function has to do some different job than the base class function.
A function has the ability to load multiple times.	A function can be overridden only a single time.
In function overloading, we don't need inheritance.	In function overriding, we need an inheritance concept.

Pure Virtual Functions and Abstract Classes in C++

A pure virtual function is a virtual function with no implementation in the base class, declared using = 0. A class with at least one pure virtual function is an abstract class that cannot be instantiated and serves as a blueprint for derived classes, which must provide their own implementation.

A class with at least one pure virtual function becomes an abstract class and Objects of abstract classes cannot be created directly.

Abstract classes are used to define interfaces and ensure common structure among derived classes.

Useful in polymorphism where different classes share the same interface but have different behaviors.

A pure virtual function forces derived classes to override it.

`virtual void draw() = 0;` declares a pure virtual function, forcing derived classes to provide their own implementation.

Parul[®]
University

NAAC
GRADE **A++**



<https://paruluniversity.ac.in/>

